⚡ ElectraAI

Architecture, Data Flow & Token Economics

A deep-dive into how every query travels through the system

PKS Research & Development | March 2026 | v1.0

Table of Contents

1. **What is ElectraAI?**
2. **The Three Core Models**
3. **Runtime Architecture — Query to Answer**
4. **Token Economics — How Groq Bills You**
5. **Setup Pipeline — One-time Data Ingestion**
6. **Component Deep Dive**
7. **Why Each Tool Was Chosen**
8. **Operating Modes — Local vs Cloud**
9. **Frequently Asked Questions**

1. What is ElectraAI?

ElectraAI is an AI-powered smart search engine and chatbot. It answers questions about your own content — whether that content lives on a website or inside PDF product catalogues. Unlike generic AI chatbots that answer from internet knowledge, ElectraAI answers exclusively from YOUR data.

Core Principle — Retrieval-Augmented Generation (RAG)

ElectraAI never guesses. It retrieves the exact relevant content from your database first, then passes that content to an LLM to generate a precise, grounded answer. This technique is called Retrieval-Augmented Generation (RAG). Without RAG, an LLM answers from its training data which may be outdated or unrelated to your products. With RAG, the LLM is grounded in your actual data every single time.

Approach	Source of Answer	Risk	ElectraAI Uses?
Plain LLM	Model training data	Hallucinations, outdated info	No
RAG (ElectraAI)	Your own database	Only what's in your data	Yes
Fine-tuned LLM	Retrained on your data	Expensive, needs retraining often	No

2. The Three Core Models

ElectraAI uses three distinct AI models, each with a completely different job. Understanding the difference is key to understanding the whole system.

Model 1 — all-MiniLM-L6-v2 (Microsoft)

Job: Understanding Meaning (Embeddings)

Converts any text into 384 numbers (a vector) that capture semantic meaning. Does NOT generate answers — only translates language into mathematics that ChromaDB can compare. Two sentences that mean the same thing produce similar vectors even if the words differ completely.

```
Input:  'What mystery books are available?'

Output: [0.23, 0.87, 0.12, 0.94 ...]  (384 numbers)


Input:  'affordable novels under ten pounds'

Output: [0.24, 0.85, 0.11, 0.93 ...]  (very similar — same meaning!)


Input:  'car engine maintenance guide'

Output: [0.91, 0.12, 0.67, 0.03 ...]  (very different meaning)
```

Property	Detail
Made by	Microsoft
Parameters	22 million (tiny and fast)
Output	384-dimension vector per input
License	Apache 2.0 — free including commercial use
Runs on	HuggingFace Router API
Tokens counted?	No — not billed in tokens

Property	Detail
Used when	Every query + during setup pipeline

Model 2 — Mistral 7B (Mistral AI)

Job: Generating Answers Locally

Reads the full prompt (system instructions + retrieved context + user question) and generates a human-like answer. Runs entirely on your Mac via Ollama — no internet required for the LLM step.

Property	Detail
Made by	Mistral AI (France)
Parameters	7 billion
License	Apache 2.0 — fully open source
Runs on	Your Mac via Ollama (completely offline)
RAM needed	~4GB
Response time	30-60 seconds on Intel Mac CPU
Used when	Local development mode only
Tokens counted?	No — runs locally, free

Model 3 — Llama 3.3 70B (Meta)

Job: Generating Answers in the Cloud

Same job as Mistral 7B but 10x bigger — 70 billion parameters vs 7 billion. Runs on Groq's custom LPU hardware delivering responses in 1-2 seconds. This is what powers ElectraAI in production on Streamlit Cloud.

Property	Detail
Made by	Meta (Facebook)
Parameters	70 billion

Property	Detail
License	Llama 3 Community License — free for most uses
Runs on	Groq API servers (LPU hardware)
Response time	1-2 seconds
Used when	Cloud production mode (Streamlit Cloud)
Tokens counted?	Yes — billed against Groq free tier

Model Comparison

	all-MiniLM-L6-v2	Mistral 7B	Llama 3.3 70B
Purpose	Text → vector	Generate answers	Generate answers
Parameters	22M	7B	70B
Runs where	HuggingFace API	Your Mac (Ollama)	Groq servers
Speed	Fast	30-60 sec	1-2 sec
Tokens billed	No	No	Yes
Mode	Always	Local only	Cloud only
Open source	Yes (Apache 2.0)	Yes (Apache 2.0)	Yes (Meta license)

3. Runtime Architecture — Query to Answer

This section traces the exact journey of a single user query from the moment they press Enter to the moment the answer appears on screen. Every system touched, every transformation applied, and every token counted is shown below.

Key Insight

Out of 6 steps in the query pipeline, only ONE step involves token billing — the Groq API call. Everything else — HuggingFace embeddings, ChromaDB search, Streamlit rendering — is completely free.

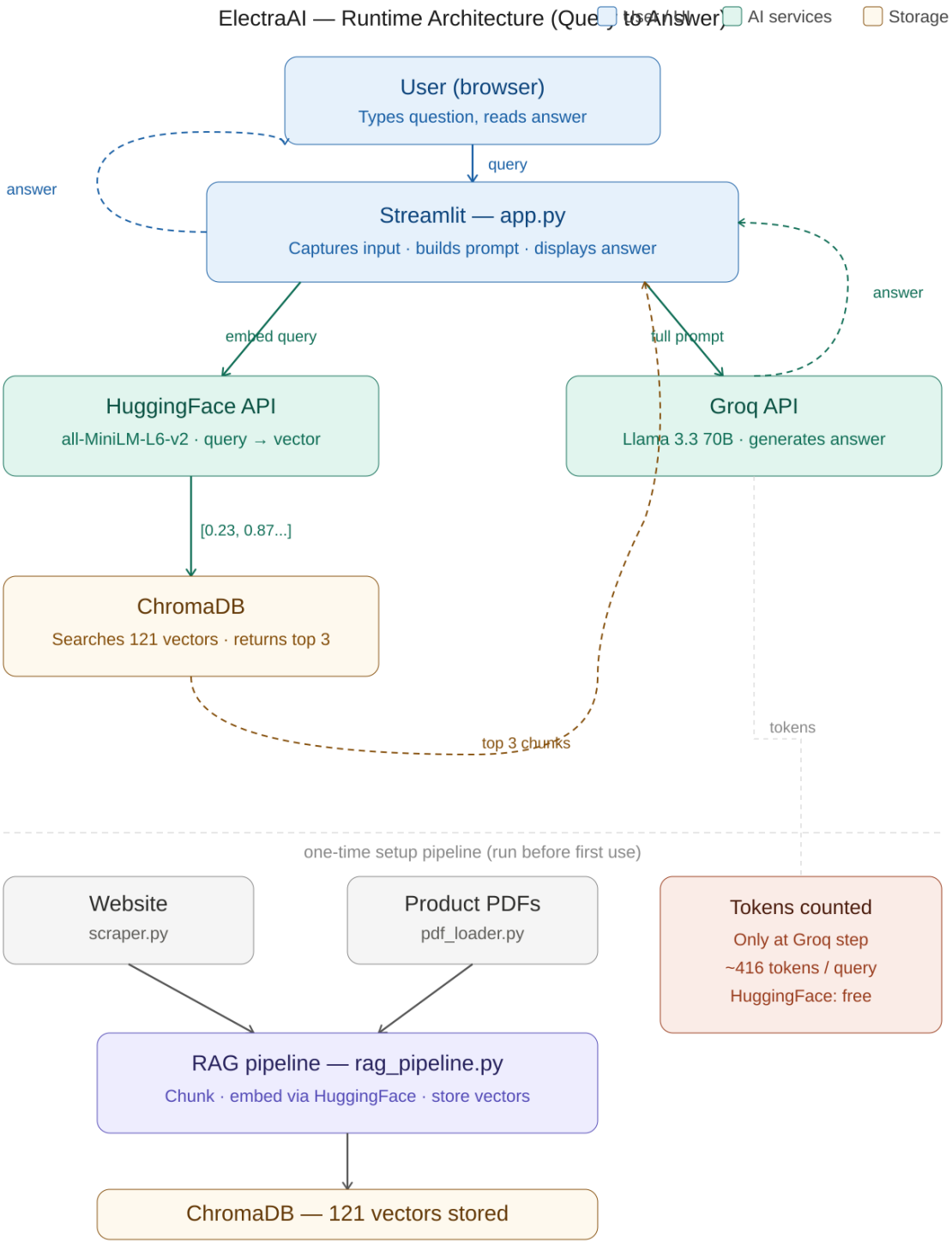

Figure 1 — ElectraAI Runtime Architecture. Top half shows the query pipeline (every query). Bottom half shows the one-time setup pipeline.

Step-by-Step Breakdown

Step 1 — User Types Query (Streamlit UI) | Tokens: 0

The user types a question into the Streamlit chat interface. Streamlit captures the input via st.chat_input() and stores it in st.session_state. No AI system is involved yet — this is purely a UI event.

Step 2 — Query Converted to Vector (HuggingFace) | Tokens: 0

The question is sent via HTTPS to the HuggingFace Router API. The all-MiniLM-L6-v2 model converts the text into a 384-dimension vector representing its meaning. This happens automatically inside retriever.invoke(prompt) — a single LangChain call that orchestrates both the embedding and the ChromaDB search.

Step 3 — Similarity Search (ChromaDB) | Tokens: 0

ChromaDB receives the query vector and performs cosine similarity search across all 121 stored vectors. It ranks every chunk by how close its meaning is to the question and returns the top 3 matches. ChromaDB is pure mathematics — no neural networks, no GPU, no billing.

Step 4 — Full Prompt Assembled (app.py) | Tokens: 0 (counted next step)

app.py combines the system prompt (~30 tokens), 3 context chunks (~300 tokens), and the user question (~6 tokens) into a single full_prompt string. This is pure Python string manipulation — no API call made yet, but this is where the token count is determined.

```
full_prompt = f"""You are ElectraAI, answer based ONLY on context below.


Context:

Source 1: {chunk_1_text}   ← ~100 tokens

Source 2: {chunk_2_text}   ← ~100 tokens

Source 3: {chunk_3_text}   ← ~100 tokens


Question: {user_question}  ← ~6 tokens

Answer:"""

# Total input: ~336 tokens sent to Groq
```

Step 5 — Answer Generated (Groq) | Tokens: ~416 BILLED HERE

The full_prompt is sent via HTTPS to Groq's servers. Llama 3.3 70B reads the entire prompt and generates an answer. Groq counts all tokens in the prompt (INPUT ~336) plus all tokens in the generated answer (OUTPUT ~80). Total: ~416 tokens deducted from your free tier quota.

Step 6 — Answer Displayed (Streamlit) | Tokens: 0

Groq returns the answer text. Streamlit renders it in the chat bubble using st.write(answer). The answer is saved to st.session_state.messages for conversation history.

Query Flow Summary Table

Step	System	Input	Output	Tokens
1	Streamlit UI	Keyboard input	Text string	0
2	HuggingFace API	Text string	384-dim vector	0
3	ChromaDB	384-dim vector	Top 3 text chunks	0
4	Python (app.py)	3 chunks + question	Full prompt string	0
5	Groq API	Full prompt (~336)	Answer text (~80)	~416

Step	System	Input	Output	Tokens
6	Streamlit UI	Answer text	Rendered chat bubble	0

4. Token Economics — How Groq Bills You

A token is not a word — it is a sub-word unit that AI models use to process text. On average, 1 token equals approximately 0.75 words or 4 characters. The diagram below shows exactly where tokens enter the picture in ElectraAI.

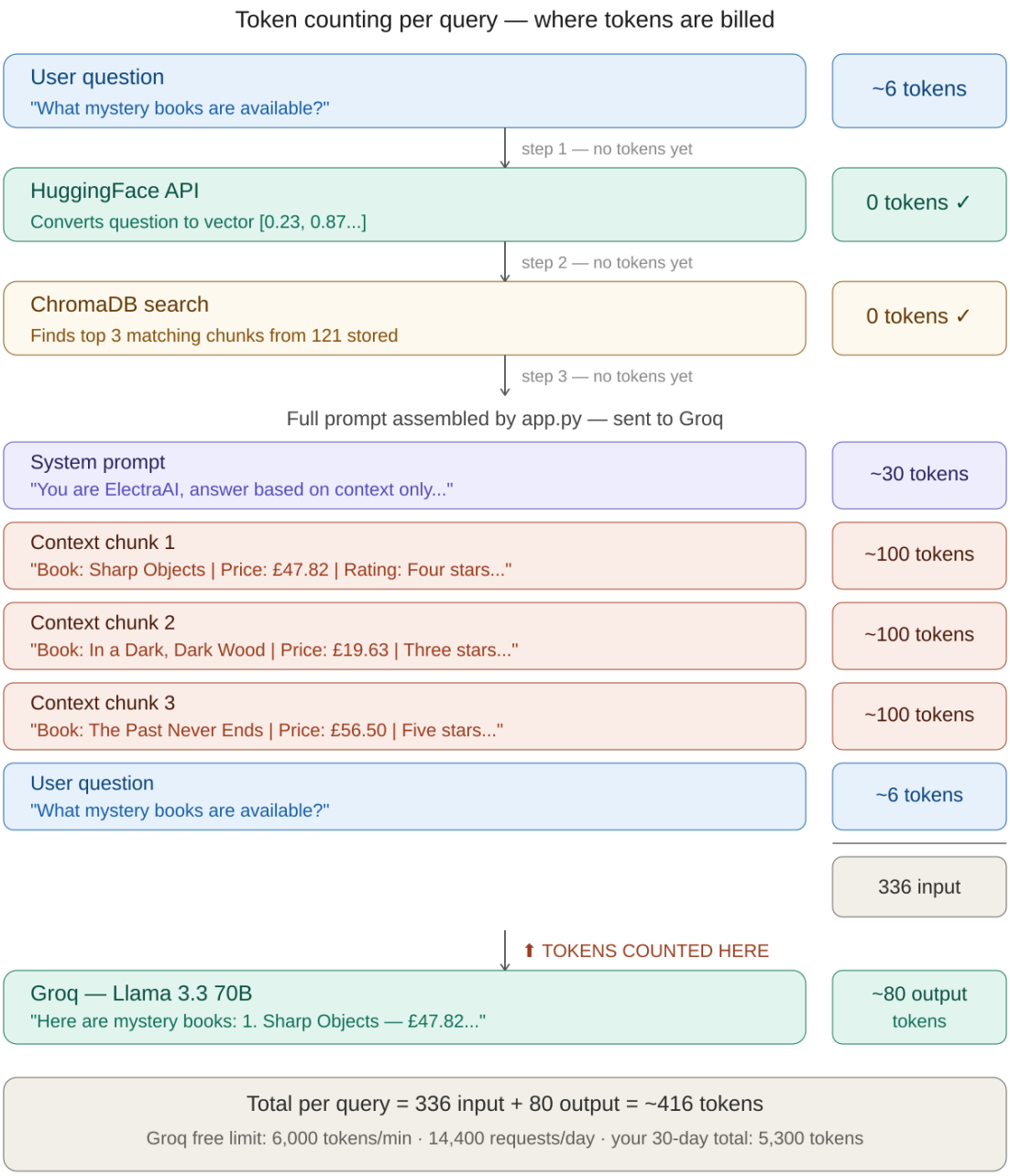

Figure 2 — Token counting per query. Only Step 5 (Groq API) bills tokens. All other steps are free.

Token Breakdown Per Query

Component	Tokens	% of Total	Can Be Reduced?
System prompt	~30	7%	Yes — shorten instructions
Context chunk 1	~100	24%	Yes — reduce chunk size
Context chunk 2	~100	24%	Yes — reduce top-K to 2
Context chunk 3	~100	24%	Yes — reduce top-K to 2
User question	~6	1%	No — user controlled
Total input	~336	80%	
Groq answer (output)	~80	20%	No — varies by answer
Total per query	~416	100%	

Groq Free Tier — Your Usage

Limit	Value	Your Usage	Headroom
Tokens per minute	6,000	~416 per query	14 queries/min max
Requests per day	14,400	~12 in 30 days	Virtually unlimited
30-day token total	No stated cap	5,300 tokens	You used 0.09% of per-minute quota
Credit card required	No	N/A	Cannot be charged accidentally

You Cannot Be Charged By Accident

Groq's free tier requires no credit card. When you exceed a rate limit, the API returns a 429 error and waits for the reset window. There is no automatic billing upgrade. Your current 30-day usage of 5,300 tokens equals approximately 0.09% of what Groq gives you every single minute.

5. Setup Pipeline — One-Time Data Ingestion

Before ElectraAI can answer questions it needs to process your source data and build the ChromaDB vector database. This pipeline runs once — or whenever your data changes. It does not run on every query.

The Four Stages

Stage	Script	Input	Output	Time
1. Collect	scraper.py or pdf_loader.py	Website URL or PDF files	website_data.json	1-5 min
2. Chunk	rag_pipeline.py	JSON with page text	121 text chunks	Seconds
3. Embed	rag_pipeline.py	121 text chunks	121 vectors (384-dim each)	2-5 min
4. Store	rag_pipeline.py	121 vectors	ChromaDB database files	Seconds

Chunking Strategy

Parameter	Value	Reasoning
Chunk size	500 characters	Small = precise retrieval; large enough for context
Chunk overlap	50 characters	Prevents information loss at boundaries
Splitter	RecursiveCharacterTextSplitter	Splits at sentence/paragraph boundaries first
Total chunks	121 (from 50 pages)	~2.4 chunks per page average

When to Re-run the Setup Pipeline

Run it when: (1) you add new PDF product catalogues, (2) your website content changes significantly, (3) you change the chunk size or embedding model. After re-running, delete the old chroma_db folder, run rag_pipeline.py, commit the new database to GitHub, and reboot the Streamlit app.

6. Component Deep Dive

Streamlit — The UI Layer

Streamlit is a Python library that converts Python scripts into interactive web applications without requiring any HTML, CSS, or JavaScript. It provides all chat UI components and serves as the orchestrator that calls all other services.

Streamlit Code	What User Sees
st.set_page_config()	Browser tab title and icon
st.title()	Large heading at top
st.success() / st.info()	Green or blue mode banner
st.chat_message()	Rounded chat bubble
st.chat_input()	Text input field at bottom
st.spinner()	Animated loading indicator
st.write(answer)	Rendered answer text
@st.cache_resource	Prevents reloading models on each query

LangChain — The Orchestration Layer

LangChain connects AI components together. Without it, integrating HuggingFace, ChromaDB, and Groq would require hundreds of lines of custom code. The most important abstraction is the retriever — one call that handles both embedding and vector search.

```
# ONE line handles both HuggingFace API call AND ChromaDB search:

docs = retriever.invoke(prompt)

#

# What LangChain does behind the scenes:

# 1. Sends prompt to HuggingFace → gets 384-dim vector back

# 2. Passes vector to ChromaDB → similarity search

# 3. Returns top 3 matching text chunks

# Without LangChain: ~50+ lines of custom integration code
```

ChromaDB — The Vector Database

ChromaDB stores embedding vectors and enables fast cosine similarity search. It persists to SQLite files on disk, making it trivial to bundle with the code and deploy anywhere — including GitHub for Streamlit Cloud access.

ChromaDB Property	Value
Storage format	SQLite (chroma.sqlite3)
Search method	Cosine similarity on 384-dim vectors
Vectors stored	121
Query result	Top 3 most similar chunks
Location (local)	database/chroma_db/ on your Mac
Location (cloud)	Committed to GitHub, loaded by Streamlit

7. Why Each Tool Was Chosen

Why Groq and not OpenAI?

Factor	Groq	OpenAI GPT-4
Cost	Free tier, no credit card	Pay per token, billing required
Speed	1-2 sec (LPU hardware)	3-10 sec (GPU)
Model quality	Llama 3.3 70B — excellent	GPT-4 — excellent
Model ownership	Open source (Meta)	Closed source (OpenAI)
Vendor lock-in	Low — swap model easily	High — OpenAI specific API

Why HuggingFace API and not Ollama for embeddings?

Factor	HuggingFace API	Ollama (local)
Works on Streamlit Cloud?	Yes — API call only	No — requires local install
Works locally?	Yes — same API	Yes
Dependencies	huggingface-hub only	Separate running process
Python 3.13 compatible?	Yes	Yes, but torch conflicts
Cost	Free tier	Free (uses local compute)

Why ChromaDB and not Pinecone?

Factor	ChromaDB	Pinecone
Runs locally?	Yes — no server needed	No — cloud only
Works on Streamlit Cloud?	Yes — files in GitHub repo	Yes — API call
Cost	Free forever	Free tier limited
Data portability	SQLite files you own	Locked to Pinecone cloud
Setup complexity	Zero — pip install only	API key + account setup

Why Streamlit and not Flask or React?

Factor	Streamlit	Flask + HTML	React
Lines of code for chat UI	~50 Python	~300+ lines	~200+ JS
Languages needed	Python only	Python + HTML + CSS	JavaScript + CSS
Built-in free hosting	Yes — Streamlit Cloud	No	No
Learning curve	1 hour	Days	Weeks

8. Operating Modes — Local vs Cloud

ElectraAI automatically detects whether it runs on your Mac or Streamlit Cloud and selects the right LLM. The same codebase, zero manual switching.

```
# Environment detection in app.py
IS_CLOUD = os.path.exists('/mount/src')  # True on Streamlit Cloud only


def get_llm():
    if GROQ_API_KEY:           # Key present → Groq cloud
        return ChatGroq(api_key=GROQ_API_KEY,
                        model_name='llama-3.3-70b-versatile')
    else:                      # No key → local Ollama
        return OllamaLLM(model='mistral')
```

Side-by-Side Comparison

Component	Local Mode	Cloud Mode
LLM	Mistral 7B via Ollama	Llama 3.3 70B via Groq
Embeddings	HuggingFace Router API	HuggingFace Router API
Vector DB	ChromaDB on local disk	ChromaDB from GitHub repo
UI URL	localhost:8501	electraai.streamlit.app
Start command	ollama serve + streamlit run	Automatic on GitHub push
Response time	30-60 seconds (Intel CPU)	1-2 seconds (Groq LPU)
Token billing	None — local compute	~416 tokens per query
Internet needed	Only for HuggingFace API	Yes — all AI via APIs

9. Frequently Asked Questions

Will Groq start charging me automatically?

No. Groq's free tier requires no credit card. Usage is capped by rate limits — when exceeded the API returns an error and resets. There is no automatic billing upgrade. Your 30-day total of 5,300 tokens is approximately 0.09% of what Groq gives you every single minute for free.

Will Streamlit Cloud keep my app running 24/7?

On the free tier, apps sleep after ~7 days of inactivity and wake in 30-60 seconds when visited. For guaranteed 24/7 uptime: Streamlit Teams (~$250/month), Railway (~$5/month), or DigitalOcean (~$12/month).

What if my query has no relevant matches in ChromaDB?

The system prompt instructs ElectraAI to say 'I don't have that information' if the answer is not in the context chunks. This prevents hallucination — ElectraAI will never invent product details not present in the database.

Can I replace the test data with real electrical product data?

Yes — that is the intended production use case. Place your PDFs in data/pdfs/ and run pdf_loader.py, or point the scraper at your website URL. Then delete chroma_db/, run rag_pipeline.py, commit to GitHub, and reboot the Streamlit app.

Why is local mode slow?

Local mode runs Mistral 7B on your Mac's Intel CPU at ~3-5 tokens per second. Cloud mode via Groq uses LPU hardware that processes hundreds of tokens per second. Apple Silicon (M1/M2/M3) Macs are significantly faster locally due to the Neural Engine.

Are all the models free and open source?

Yes. all-MiniLM-L6-v2 (Apache 2.0), Mistral 7B (Apache 2.0), and Llama 3.3 70B (Meta Community License) are all open weights models. Groq and HuggingFace charge for API usage above free tier thresholds but both have generous free tiers. The total licensing cost of the models themselves is zero.

⚡ ElectraAI Architecture Documentation v1.0

PKS Research & Development | March 2026

Groq + HuggingFace + ChromaDB + LangChain + Streamlit